

École Polytechnique Fédérale de Lausanne

Exploiting simultaneous image and LIDAR data for point-cloud
classification

Master Thesis

by Guillaume Spahr

Prof. Dr. Pierre Vanderghenst
Professor

Dr. Nicolas Aspert
Co-advisor

Laboratoire du Traitement du Signal 2
EPFL

June 26, 2026

Abstract

Semantic segmentation of 3D point clouds plays a great role in understanding large-scale scenes, yet it remains a difficult problem. LiDAR¹ data is inherently sparse and irregular, and on its own carries limited appearance information, which makes accurate classification of scene elements genuinely hard. Point-based and transformer-based backbones have made impressive improvements on several benchmarks in recent years, and a growing number of work has shown that pairing LiDAR with camera images can meaningfully boost 3D predictions.

This thesis investigates how visual features extracted from calibrated images can be integrated into a lightweight 3D backbone for point cloud segmentation. Starting from recent point transformer architectures, we explore the use of frozen vision foundation models as a source of dense image features, and examine how information from these models can be injected into point-wise representations. More specifically, we look at extracting features from multiple layers of such models and design fusion modules that can aggregate visual cues from one or several camera views associated with each 3D point.

We evaluate our methods on two outdoor segmentation benchmarks and on two aerial datasets acquired by Helimap, comprising high-resolution LiDAR scans and calibrated imagery from real-world environments.

Index Terms

LiDAR, Images, Segmentation

DISCLAIMER ABOUT AI USAGE

Generative AI was used to rewrite and reformulate the phrasing of some sentences in order to enhance the clarity of the content. The work, figures, graphs, results, analyses, and conclusions were entirely produced by me and remain my own responsibility.

I. INTRODUCTION

Point clouds have become a fundamental data representation in many applications, including autonomous driving, robotics, augmented reality, infrastructure inspection, terrain mapping, medical imaging, and general 3D scene understanding [1]–[4]. They represent objects or environments as sets of points in three-dimensional space, often enriched with additional attributes such as color, intensity or surface normals. This representation is particularly effective for capturing the geometry of real-world scenes, since it directly describes the spatial structure of objects and surfaces.

However, point clouds also differ significantly from images. While images are defined on a regular two-dimensional grid, point clouds are unordered, irregular, sparse, and often noisy [5]–[7]. As a result, standard convolutional operations designed for images cannot be directly applied to raw point sets. Moreover, the density of a point cloud may vary strongly depending on the sensor, the distance to the observed surface, the acquisition geometry, and the preprocessing pipeline. These properties make point cloud understanding a challenging problem, especially for large-scale outdoor scenes.

Among the different tasks involving point clouds, semantic segmentation is one of the most important and widely studied [2], [8]–[10]. Its goal is to assign a semantic label to each point of the scene. This requires fine-grained 3D understanding, since the model must not only recognize individual objects or structures, but also correctly delineate their spatial extent. Accurate point cloud segmentation is essential for many applications that rely on detailed spatial reasoning, such as autonomous navigation, pedestrian detection, object separation in cluttered environments, railway and road infrastructure monitoring, industrial inspection, and terrain analysis.

In recent years, deep learning has led to substantial progress in point cloud segmentation. Early point-based methods process raw point sets directly [5], [6], while sparse convolutional architectures [11], [12], graph-based and kernel-based operators [13], [14], and recent transformer-based backbones [15], [16] have progressively improved accuracy and scalability.

Despite these advances, point cloud segmentation remains difficult in realistic environments. LiDAR point clouds provide accurate geometric information, but they are usually sparse compared to images and may lack detailed appearance cues. This can make it difficult to distinguish classes with similar geometry

¹Light Detection and Ranging

but different visual appearance. For example, road surfaces, sidewalks, vegetation, roofs, poles, and other infrastructure elements may present ambiguous geometric patterns depending on the acquisition setting. In such cases, complementary image information can help disambiguate the scene.

A natural way to improve 3D perception is therefore to exploit multiple sensing modalities. In LiDAR-based perception, camera images are especially valuable because they provide dense color, texture, and semantic information that complements the sparse geometric structure of point clouds [17], [18]. When the LiDAR sensor and the cameras are calibrated, each 3D point can be projected onto the image plane and associated with visual features extracted from the corresponding image region. This enables the integration of 2D appearance information into 3D point-wise representations.

Recent works have shown that pretrained vision foundation models can provide powerful visual features for downstream perception tasks [19]–[22]. Since these models are trained on large-scale image datasets, they can extract rich semantic representations without requiring additional annotation of the target dataset. This is particularly attractive for point cloud segmentation, where annotated 3D data is expensive and time-consuming to obtain. Instead of training a full image backbone jointly with the 3D network, frozen visual features can be extracted and injected into the point cloud model at different stages of the architecture.

However, the fusion of image and point cloud information remains a non-trivial problem. First, the two modalities have different structures: images are dense and regular, while point clouds are sparse and irregular. Second, the correspondence between 3D points and image pixels depends on camera calibration, visibility, occlusions, and discretization effects caused by image patching or feature-map downsampling. Third, in multi-camera or multi-view settings, a single 3D point may be visible in several images. These different viewpoints may provide complementary information, but they may also vary in reliability depending on the viewing angle, occlusions, distance, or image quality [23]. Therefore, simply assigning one image feature to each 3D point may not fully exploit the available visual information.

This work focuses on semantic segmentation of LiDAR point clouds with additional image information. We build upon recent lightweight point transformer architectures [15], [16] and investigate how pretrained image backbones [20], [21] can be used to enrich 3D point features. In particular, we study two image feature fusion modules that incorporate multi-layer visual representations into the 3D decoder. We also investigate a multi-camera attention mechanism that aggregates several image observations associated with the same 3D point and learns to weight the relative importance of each camera view.

The study is conducted in collaboration with Helimap², which provided two large-scale aerial datasets with annotated LiDAR point clouds and calibrated images. These datasets allow us to evaluate the proposed approach in realistic outdoor acquisition settings, including a railway station and a highway scene. In addition, we compare our method on two public outdoor benchmarks [2], [10] in order to assess its behavior beyond the Helimap data.

The main contributions of this work can be summarized as follows:

- We investigate image-assisted point cloud segmentation by combining a lightweight 3D backbone with frozen vision foundation model features.
- We design and evaluate two multi-layer image feature fusion modules, based on feature concatenation and spatial pyramid representations, for injecting visual information into the 3D decoder.
- We introduce a multi-camera attention mechanism to aggregate multiple image observations of the same 3D point.
- We evaluate the proposed approach on public outdoor datasets and on two high-resolution aerial datasets provided by Helimap.

II. RELATED WORK

3D Backbones for Semantic Segmentation

Point cloud processing methods can be broadly categorized into three main paradigms:

- point-based;
- voxel-based;
- and projection-based approaches.

²<https://helimap.ch/fr/home-fr/>

Projection-based methods consist of mapping 3D point clouds onto one or multiple 2D planes, effectively creating image-like representations. These projections can then be processed using 2D convolutional neural networks (CNNs), which are computationally efficient and highly optimized for modern GPU architectures [7]. However, this dimensionality reduction introduces ambiguity, as part of the original 3D geometric structure is lost or only implicitly preserved through multi-view consistency.

Voxel-based methods discretize the 3D space into a regular grid of volumetric cells (voxels), enabling the use of 3D convolutions similarly to standard CNNs. A key challenge in this setting is the sparsity of point clouds, which makes dense 3D convolution computationally inefficient. To address this, sparse convolution techniques have been proposed, allowing computation to be restricted to occupied regions only [11], [12]. An important design choice in voxel-based methods is the voxel resolution. While higher resolutions improve geometric fidelity, they also increase computational cost and limit the effective receptive field, as scaling kernel sizes becomes prohibitive.

Point-based methods operate directly on raw point sets without imposing any discretization. Since point clouds are inherently unordered and irregular, standard convolution operations cannot be directly applied. Instead, these methods rely on permutation-invariant functions. A seminal work in this direction is PointNet [5], which uses shared multi-layer perceptrons (MLPs) combined with a symmetric aggregation function to ensure invariance to input permutations. It also incorporates a learned spatial transformation module to align input points. Subsequent extensions such as PointNet++ [6] introduce hierarchical grouping strategies to capture local geometric structures. Other works further explore local neighborhood modeling through graph-based or kernel-based operations [13], [14].

More recently, transformer-based architectures have been introduced to model long-range dependencies in point clouds. These models leverage self-attention mechanisms but suffer from quadratic complexity with respect to the number of points. The recent Point Transformer V3 (PTv3) [15] demonstrates that competitive performance can be achieved with a scalable architecture. This model reduces computational and memory overhead by removing expensive k-nearest neighbors (k-NN) search and complex architectural attention operations. Instead, it introduces a serialization strategy based on space-filling curves, which imposes an ordering over the point cloud while preserving spatial locality. This enables efficient grouping of neighboring points and the application of localized self-attention within small patches.

The PTv3 architecture follows a U-Net [24] design, allowing multi-scale feature extraction and precise spatial reconstruction. Recent works have also explored self-supervised pretraining strategies for point cloud models [25], [26]. For instance, Utonia [27] aims to learn a general-purpose representation that can generalize across datasets and remain robust to missing input features.

Building on PTv3, LitePT [16] proposes several architectural simplifications and introduces an efficient relative positional encoding mechanism inspired by RoFormer [28]. These improvements lead to better performance while reducing both computational and memory requirements. For this reason, we use LitePT as the main 3D backbone in this work. Its efficiency makes it a suitable choice for image-assisted segmentation, where the integration of additional 2D features increases the overall memory and computational requirements.

2D Backbones and Visual Feature Extraction

Images provide dense appearance information that can complement the sparse geometric structure of point clouds. In particular, color, texture, object boundaries, and semantic context can help disambiguate classes that are difficult to separate from geometry alone. For this reason, the choice of the image backbone is important in image-assisted point cloud segmentation.

A straightforward option is to train a 2D network jointly with the 3D backbone. However, modern 3D segmentation models are already computationally demanding, especially for large-scale outdoor point clouds. Adding a trainable image branch can therefore substantially increase memory usage and training time. An alternative is to use pretrained vision models as frozen feature extractors. This strategy makes it possible to benefit from rich visual representations without fine-tuning the image backbone.

Classical visual backbones are often based on convolutional neural networks (CNNs). A representative example is ResNet [29], which introduced residual connections and became a standard architecture for visual feature extraction. CNNs naturally build hierarchical representations through successive downsampling stages, making them well suited for dense prediction tasks that require both local details and high-level semantics.

More recently, Vision Transformers (ViTs) [30] have become a strong alternative to CNNs. Instead of relying on convolutional filters, ViTs divide the image into patches and process the resulting tokens using self-attention. This design allows the model to capture long-range interactions between image regions. However, standard ViTs typically preserve a fixed spatial token resolution across layers and do not naturally produce the multi-scale feature hierarchy found in CNNs. This can be limiting for dense prediction tasks, where both fine spatial details and global semantic context are important.

Self-supervised learning has further increased the interest in ViT-based image backbones. The DINO family has shown that strong visual representations can be learned without manual labels. DINO [19], DINOv2 [20], and DINOv3 [21] learn transferable features that can be reused in downstream tasks. In the context of 3D segmentation, such features are particularly attractive because they can be extracted from calibrated images and projected onto point clouds, as demonstrated by recent image-assisted 3D segmentation methods [31].

Although DINO features provide rich semantic information, they are not explicitly optimized for geometric reasoning. This motivates the exploration of additional vision models trained for geometry-related tasks, such as monocular depth estimation, camera pose estimation, or dense 3D reconstruction. Models such as DUST3R [32], VGGT [33], and Depth Anything V2 [34] are examples of this direction. Depth-oriented features may provide complementary cues to semantic image features, although their usefulness depends on the target dataset, the quality of the image-point correspondence, and the fusion strategy.

Another important question is how to extract and combine features from different layers of an image backbone. Earlier layers tend to preserve more local and spatially detailed information, while deeper layers usually encode more semantic and abstract representations [35]. Dense prediction architectures such as Dense Prediction Transformer (DPT) [36], SegDINO [37], and Feature Pyramid Networks (FPN) [38] exploit this idea by combining intermediate features and, in some cases, organizing them into multi-scale representations through top-down refinement.

In this work, we mainly rely on DINOv3 as a frozen image backbone, while keeping the proposed fusion modules compatible with other pretrained visual encoders. Our design is motivated by the observation that different layers of a visual backbone encode complementary information. We therefore investigate multi-layer feature fusion and spatial pyramid representations before injecting image features into the 3D decoder.

Fusion Mechanisms

A large body of work has explored the fusion of point clouds and images for 3D scene understanding. These methods are commonly categorized according to the stage at which the modalities interact [39]. Early fusion combines image and point cloud information close to the input representation, for instance by appending projected image features or color values to point features. Late fusion processes each modality independently and combines the final predictions, typically through averaging, voting, or learned decision-level aggregation. Intermediate fusion lies between these two strategies: each modality is first processed by a dedicated backbone, and the resulting feature representations are then combined at one or several stages of the network. This strategy is particularly attractive for semantic segmentation, since it preserves modality-specific feature extraction while allowing visual and geometric information to interact before the final prediction.

Robert et al. [23] proposed a multi-view 2D–3D fusion method for large-scale semantic segmentation. In their approach, each 3D point can aggregate image features from multiple views. The authors compute viewing-condition descriptors for each point-view pair and use them to predict attention weights over the available images. A gating mechanism is also introduced to suppress unreliable visual features. The resulting weighted image representation is then fused with point cloud features at different stages of the 3D network, enabling early, intermediate, or late fusion. This work highlights the importance of view selection and reliability estimation when multiple image observations are available for the same 3D point.

More recently, DINO in the Room [31] showed that pretrained vision foundation models can provide strong visual features for 3D semantic segmentation. Their method associates 3D points with image pixels through geometric reprojection, allowing each point to inherit image features extracted from corresponding regions. These features are then injected into a 3D backbone, and the authors further show that distilling DINO features into the 3D network can serve as an effective pretraining strategy. This demonstrates that frozen 2D foundation models can provide useful semantic cues for 3D point-wise prediction.

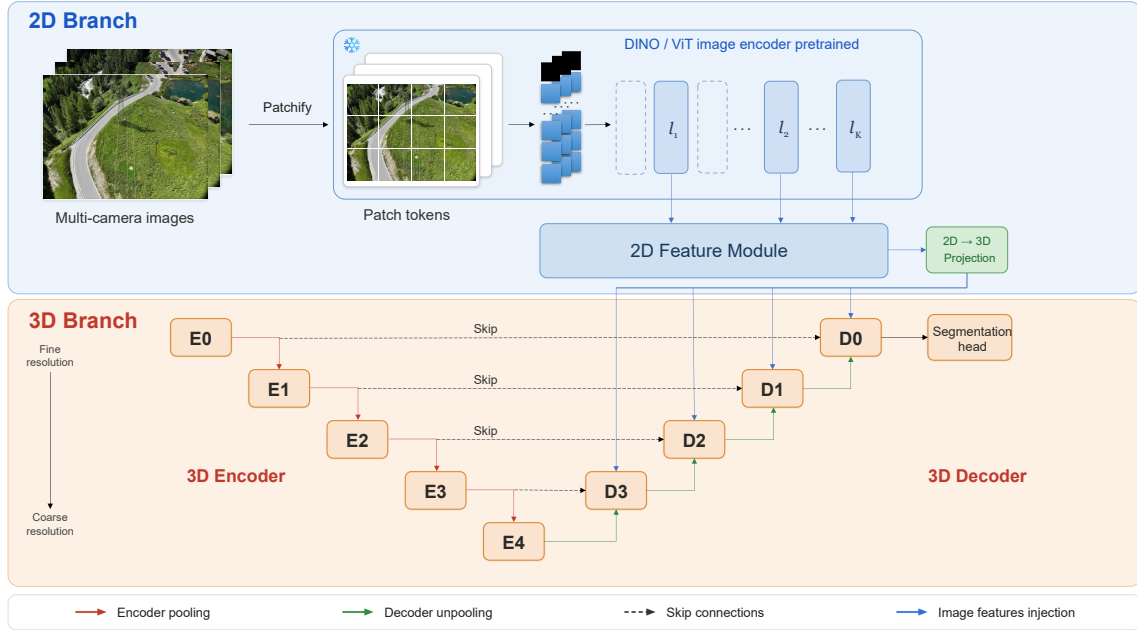


Fig. 1: **Model architecture:** Model architecture. The model consists of two branches: a 3D branch and a 2D branch. The 3D branch is based on a LitePT [16] backbone with a U-Net architecture, processing raw point cloud features (coordinates, colors, normals, or intensity) and progressively extracting higher-level semantic information. The 2D branch extracts visual features from a pretrained DINO/ViT [21], [30] backbone, using both the final-layer output and a set of intermediate layers $l_1, l_2, \dots, l_K$. In our experiments, $K=4$, with layers uniformly sampled across the ViT depth. These features are refined by a 2D feature module and projected onto the point cloud through offline pairing, enabling multimodal fusion in the decoder of the 3D backbone.

However, multi-view fusion remains challenging. In realistic acquisition settings, a 3D point may be visible in several images, but the corresponding views are not necessarily equally informative. Some views may be affected by occlusions, unfavorable viewing angles, motion, calibration noise, image blur, or discretization errors caused by feature-map downsampling. Therefore, directly selecting one view, averaging several views, or concatenating all projected features may not fully exploit the available visual information. A fusion module should ideally be able to reason about the relative reliability and complementarity of each camera observation.

In this work, we focus on intermediate fusion between a lightweight 3D backbone and frozen image features. Building on the idea that different camera observations may provide complementary evidence, we investigate a multi-camera attention mechanism that aggregates several image features associated with the same 3D point. Instead of treating all views equally, the proposed module learns to weight image observations according to their relevance for the current point representation. This design aims to improve robustness to occlusions, projection noise, and viewpoint-dependent ambiguities while preserving the efficiency required for large-scale point cloud segmentation.

III. METHOD

Our general architecture is illustrated in Figure 1. Given a point cloud and a set of calibrated images, our method extracts 3D features using a LitePT backbone and visual features using a frozen DINO image encoder. The camera calibration parameters are then used to associate 3D points with image tokens, allowing visual features to be injected into selected decoder stages of the 3D network. We investigate two image feature modules for multi-layer visual representation and a multi-camera attention module for aggregating visual observations from multiple cameras associated with the same 3D point.

We now describe the proposed method in detail. We first introduce the 3D–2D projection used to establish point-image correspondences, before presenting the 3D backbone, the image feature modules, and the proposed fusion mechanisms.

A. Point-Image pairing

A 3D point in world coordinates is denoted by $P_w = [X \ Y \ Z]^T$. To project this point onto the image plane, it must first be expressed in the camera coordinate system. This transformation is performed using the camera rotation matrix R and translation vector t . Under the pinhole camera model, the point in camera coordinates is given by

$$P_c = \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = [R \mid t] \hat{P}_w,$$

where $\hat{P}_w = [X \ Y \ Z \ 1]^T$ denotes the homogeneous representation of P_w .

The corresponding image coordinates are then obtained by applying the camera’s intrinsic matrix K :

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ s \end{bmatrix} = K \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix},$$

where s is the projective scale factor, typically equal to Z_c depending on the camera convention. The pixel coordinates are obtained by normalization:

$$u = \frac{x}{s}, \quad v = \frac{y}{s}.$$

This corresponds to the standard pinhole camera formulation. A more detailed projection model, including the Agisoft Metashape convention and lens distortion terms, is reported in Appendix VI following [10].

A point is considered to be observed by an image if its projected coordinates lie within the image boundaries and if it is located in front of the camera, i.e. $Z_c > 0$ for this formulation.

To reduce occlusion artifacts, where points may project onto the image plane despite being hidden behind closer surfaces, a simple z-buffer strategy is used. For each pixel (i, j) , we define the set of 3D points projected onto that pixel as

$$\mathcal{P}_{i,j} = \{p \in \mathcal{P} \mid \pi(p) = (i, j)\},$$

where $\mathcal{P}$ denotes the point cloud and $\pi(p)$ is the projection of point p onto the image plane. The depth map is then constructed by retaining the closest point among this set:

$$D(i, j) = \min_{p \in \mathcal{P}_{i,j}} Z_c(p).$$

A projected point $p \in \mathcal{P}_{i,j}$ is considered visible if

$$|Z_c(p) - D(i, j)| < \tau,$$

where τ is a depth tolerance threshold.

The image features used for fusion are extracted from intermediate feature maps of a CNN or Vision Transformer. These feature maps generally have a lower spatial resolution than the original image. For a ViT with patch size or effective stride (P), the projected pixel coordinate (u, v) is mapped to a feature-map index by

$$(\hat{u}, \hat{v}) = \left(\left\lfloor \frac{u}{P} \right\rfloor, \left\lfloor \frac{v}{P} \right\rfloor \right),$$

where $\lfloor \cdot \rfloor$ denotes the floor function. This follows the projection procedure used in [31]. When images are resized or cropped before being processed by the image backbone, the same transformation is applied to the projected pixel coordinates before computing the feature-map index.

This procedure produces, for each 3D point, a set of valid image-token associations. A point may be observed in no image, in a single image, or in multiple images. These associations are used in the following sections to inject visual features into the 3D decoder and to aggregate multi-view information.

B. 3D Backbone

We use LitePT [16] as the 3D backbone of our architecture. LitePT is a lightweight point transformer designed for efficient point cloud segmentation. It combines sparse convolutional operations with transformer-based attention blocks, offering a favorable trade-off between segmentation accuracy, memory consumption, and inference speed. This efficiency is particularly important in our setting, since the 3D backbone is extended with additional image features and fusion modules.

The input to the 3D branch, shown in orange in Figure 1, is a point cloud with spatial coordinates and, depending on the dataset, optional point attributes such as color, intensity, normals, or other descriptors. The coordinates are used to voxel-quantize the point cloud: each continuous coordinate $\mathbf{x}$ is divided by a fixed voxel size g and floored, $\mathbf{v} = \lfloor \mathbf{x}/g \rfloor$, mapping every point to a discrete cell of a regular 3D grid. Points that fall in the same cell are reduced to a single representative, and since the occupied voxels form only a small fraction of the grid, only they are stored, yielding a sparse representation. This representation is then processed by an initial sparse 3D convolutional stem, which extracts low-level point features efficiently.

The encoder follows a hierarchical multi-scale structure. At each stage, point features are processed by sparse convolutional blocks, attention-based blocks, or hybrid blocks depending on the LitePT configuration. Between consecutive stages, the point representation is downsampled through grid pooling. Points falling into the same coarser voxel are grouped together and their features are aggregated, producing a reduced set of points at the next scale. This progressively decreases the spatial resolution while increasing the receptive field of the network. The encoder also stores intermediate features as skip connections, which are later used by the decoder to recover spatial details.

The decoder reconstructs high-resolution point-wise representations in a symmetric manner. At each decoder stage, features from the coarser level are unpooled to the corresponding finer point set and combined with the skip features from the encoder. The resulting representation is then refined by the decoder block at that stage. In the LitePT variant used in this work, the coarsest decoder stage contains an attention block, while the remaining decoder stages are based on sparse convolutional blocks. This design preserves the efficiency of the decoder while still allowing long-range interactions at the coarsest resolution.

In our multimodal architecture, the decoder is also the main location where image information is introduced. Image features projected from calibrated views are injected at selected decoder stages, after the 3D features have been aligned with the point set of the current resolution. This allows the network to refine point-wise representations using both geometric information from the 3D backbone and semantic cues from the image branch. The exact fusion mechanism is described in the following sections.

C. 2D Backbone and Multi-Layer Feature Fusion

For image feature extraction, we use a frozen DINOv3 image encoder. Although our experiments mainly rely on DINOv3, the proposed fusion modules are not specific to this backbone and can also be applied to DINOv2 or other pretrained visual encoders. For the DINOv3 variant used in our experiments, the image is divided into patches with an effective stride of ($P=16$). From the image backbone, we extract a set of (K) intermediate feature maps from different Transformer layers. These feature maps are defined on the same native token grid, but correspond to different depths of the network and therefore encode complementary visual information. Earlier layers tend to preserve more local visual patterns and spatial details, while deeper layers generally contain more abstract and semantic information. This multi-layer representation is particularly interesting for point cloud segmentation, where fine spatial boundaries and high-level semantic cues are both important.

Since the 3D backbone also processes information at multiple encoder-decoder stages, we investigate whether multi-level 2D representations can improve the 3D feature hierarchy. To this end, we consider two image feature modules inspired by dense prediction architectures such as Dense Prediction Transformer (DPT) [36], SegDINO [37], and Feature Pyramid Networks (FPN) [38]. The main idea is to expose the 3D decoder not only to the final output of the image encoder, but also to intermediate visual features that may contain complementary local and semantic information.

The first architecture is a lightweight multi-layer concatenation module. Each selected 2D feature map is independently passed through a DINO projector composed of channel-wise layer normalization, a (1×1)

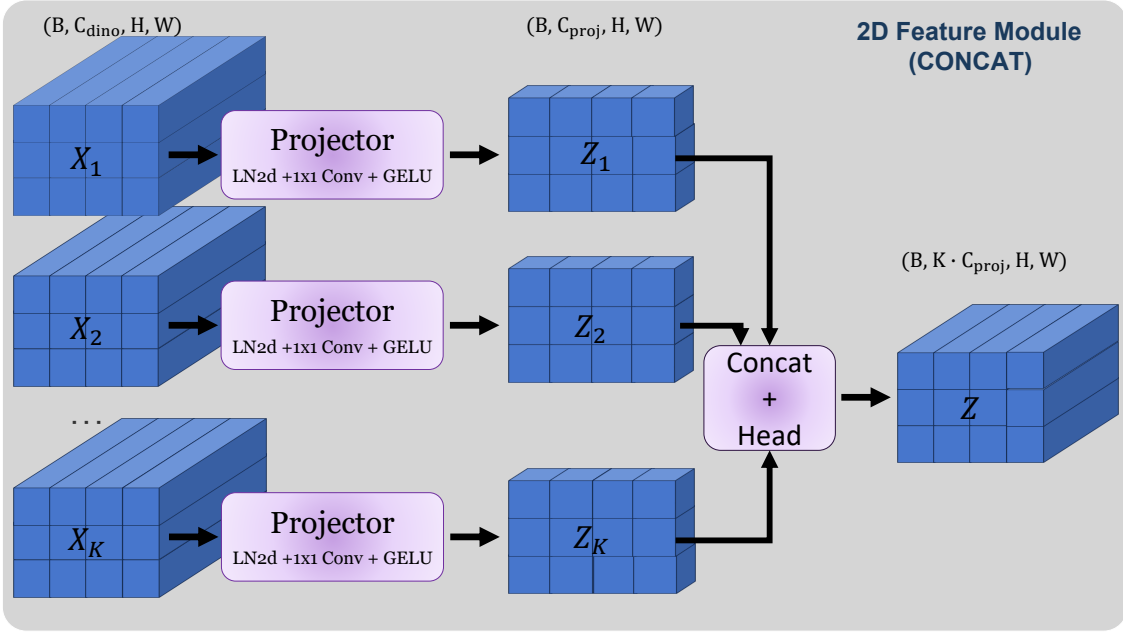


Fig. 2: **Concatenation module:** The (K) intermediate feature maps X_i are individually projected by their corresponding "Projector" module composed on linear normalization modules, pointwise convolution and a GELU activation to obtain the (K) projected maps. They act as dimensionality reducer (i.e. $C_{proj} < C_{dino}$). They are then concatenated and a final head is used to further refine the features. This creates a single output with dimension $K \cdot C_{proj}$. It is simpler than the pyramid variant and do not reduce the spatial resolution.

convolution, and a GELU activation [40]. This projection maps the original DINO feature dimension C_{dino} to a lower-dimensional representation C_{proj} , while preserving the native spatial token grid. The projected feature maps are then concatenated along the channel dimension, producing a fused representation with $K \cdot C_{proj}$ channels. Finally, a (1×1) convolution followed by layer normalization is applied to produce the output image feature map. This design introduces only a small computational and memory overhead, while exposing multiple levels of abstraction from the 2D backbone to the 3D model. A schematic illustration of this module is provided in Figure 2.

The second architecture extends this idea by constructing a spatial pyramid of 2D features aligned with the multi-scale hierarchy of the 3D decoder. Its architecture is shown in Figure 3. As in the concatenation module, each selected DINO feature map is first processed independently by a projector. Then, for each decoder stage (s) , all projected feature maps are resized to a stage-specific spatial resolution. In practice, we use adaptive average pooling to obtain a resolution approximately corresponding to the downsampling level d_s of the 3D stage. The finest stage keeps the native token resolution, while deeper stages receive progressively coarser image feature maps. This design makes it possible to inject visual information at different spatial scales, in a way that is consistent with the resolution of the corresponding 3D decoder stage.

For each stage (s) , the K resized feature maps are concatenated along the channel dimension. This yields a stage-specific representation

$$\mathbf{X}_s \in \mathbb{R}^{H_s \times W_s \times K C_{proj}},$$

where H_s and W_s denote the spatial resolution of the corresponding pyramid level. In contrast to weighted summation, channel-wise concatenation preserves the feature subspace of each selected 2D layer. Subsequent pointwise convolutions [41] can then learn flexible cross-layer and cross-channel interactions. This is useful because different DINO layers may encode different types of information, and forcing them to be merged too early by summation may remove part of this complementarity.

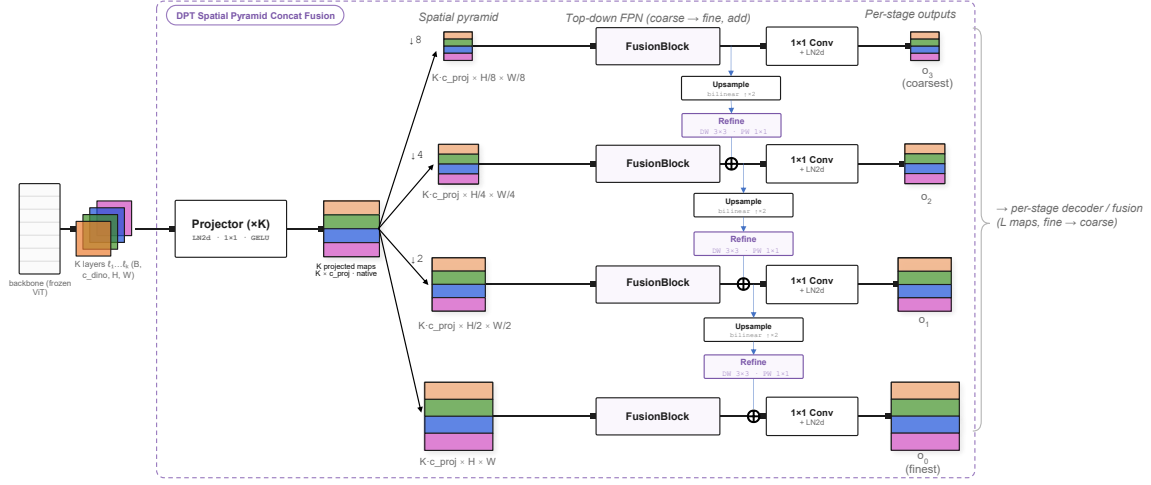


Fig. 3: **Spatial pyramid module.** Intermediate DINO feature maps are first projected to a lower-dimensional representation. For each decoder stage, the projected features are resized to the corresponding spatial resolution and concatenated along the channel dimension. A lightweight fusion block refines each stage-specific representation. A top-down refinement path then propagates coarse semantic information toward finer resolutions before the final stage-specific heads produce the image features used for 2D–3D fusion.

Each stage-specific representation is processed by a lightweight residual fusion block composed of layer normalization, a depthwise (3×3) convolution [42], a GELU activation [40], and a pointwise (1×1) convolution [41]. This block can be written as

$$\mathbf{X}'_s = \mathbf{X}_s + \text{Conv}_{1 \times 1}(\sigma(\text{DWConv}_{3 \times 3}(\text{LN}(\mathbf{X}_s)))) ,$$

where σ denotes the GELU activation. The depthwise convolution introduces local spatial mixing on the 2D token grid, while the pointwise convolution mixes the concatenated channel features. This separation keeps the block computationally light, since spatial mixing and channel mixing are performed separately. A stage-specific final head, composed of a (1×1) convolution and layer normalization, then produces the image feature map injected into the corresponding stage of the 3D decoder.

In addition, we use a top-down refinement path inspired by FPN [38] and DPT [36]. This path is applied after the stage-wise pyramid has been constructed. The coarsest feature map is processed first and then upsampled to the spatial resolution of the next finer stage using bilinear interpolation. The upsampled representation is refined with a depthwise (3×3) convolution [42], a GELU activation [40], and a pointwise (1×1) convolution [41] before being added to the finer-resolution feature map. This process is repeated until the finest stage is reached. The top-down path therefore propagates semantic information from coarse, low-resolution image features toward finer, higher-resolution stages. This is especially useful when injecting image features into a multi-scale 3D decoder, since coarser stages may benefit from more global semantic context while finer stages require more spatially detailed information.

We also experimented with a similar principle using a ConvNeXt [43] variant distilled from the large DINOv3 teacher model. This alternative is motivated by the fact that convolutional architectures naturally produce hierarchical feature maps at multiple spatial resolutions, which could in principle be well aligned with the multi-scale structure of the 3D decoder. However, in our experiments, the ConvNeXt-based variant did not improve over the Vision Transformer backbone. We therefore focus the remaining experiments on the DINOv3 ViT features.

Overall, the concatenation module provides a compact single-scale fusion of intermediate DINO features, while the spatial pyramid module produces stage-specific image representations aligned with the decoder

hierarchy of the 3D backbone. These image features are then projected onto the point cloud and injected into the corresponding 3D decoder stages.

D. Point-Image Fusion

At each decoder stage (s), the 3D backbone produces point features

$$\mathbf{F}_s \in \mathbb{R}^{N_s \times D_s}, \quad (1)$$

where N_s is the number of points or superpoints at that stage and D_s is the corresponding feature dimension. The associated image feature map is denoted by

$$\mathbf{I}_s \in \mathbb{R}^{H_s \times W_s \times C_s}, \quad (2)$$

where H_s , W_s , and C_s are the spatial resolution and channel dimension of the image representation used at stage (s). For the concatenation module, this corresponds to the native image-token resolution, while for the spatial pyramid module it corresponds to the stage-specific feature resolution described previously.

In the standard 3D decoder, features from the coarser level are unpooled and combined with the corresponding encoder skip connection. This can be written as

$$\mathbf{F}'_s = f_s^{\text{skip}}(\mathbf{F}_s) + f_s^{\text{unpool}}(\mathbf{F}_{s+1}), \quad (3)$$

where $\mathbf{F}_{s+1}$ denotes the coarser-stage features that are unpooled to match the point set of stage (s). Following DINO in the Room [31], projected image features can be injected into the decoder through an additional image branch:

$$\mathbf{F}'_s = f_s^{\text{skip}}(\mathbf{F}_s) + f_s^{\text{unpool}}(\mathbf{F}_{s+1}) + f_s^{2D}(\mathbf{I}_s). \quad (4)$$

This formulation is simple and efficient, but it assumes that the image feature associated with each point has already been selected or aggregated. In a multi-camera setting, this may be restrictive, since a single 3D point can be visible in several images and the corresponding observations are not necessarily equally reliable.

To aggregate multiple image observations, we use a cross-attention mechanism based on scaled dot-product attention [44]. Let $\mathbf{q}_s^i \in \mathbb{R}^{D'_s}$ be the feature of point (i) at decoder stage (s), obtained after the standard 3D decoder update or after a learned projection of the point features. Let

$$\mathbf{I}_s^i \in \mathbb{R}^{M \times C_s} \quad (5)$$

be the image features associated with point (i), where M is the maximum number of camera observations. Invalid camera slots are handled with a boolean validity mask.

The point feature is projected to a query, while the associated image features are projected to keys and values:

$$\mathbf{Q}_s^i = \mathbf{q}_s^i \mathbf{W}_s^Q \in \mathbb{R}^{1 \times d}, \quad \mathbf{K}_s^i = \mathbf{I}_s^i \mathbf{W}_s^K \in \mathbb{R}^{M \times d}, \quad \mathbf{V}_s^i = \mathbf{I}_s^i \mathbf{W}_s^V \in \mathbb{R}^{M \times d}. \quad (6)$$

The multi-camera image representation for point (i) is then computed as

$$\mathbf{h}_s^i = \text{softmax} \left(\frac{\mathbf{Q}_s^i (\mathbf{K}_s^i)^\top}{\sqrt{d}} + \mathbf{B}_s^i \right) \mathbf{V}_s^i \in \mathbb{R}^{1 \times d}, \quad (7)$$

where $\mathbf{B}_s^i \in \mathbb{R}^{1 \times M}$ is an attention bias used to mask invalid camera observations. Invalid views are assigned a value of $-\infty$ before the softmax, ensuring that they receive zero attention weight. The resulting vector $\mathbf{h}_s^i$ is projected back to the decoder feature dimension and injected into the 3D feature stream.

Using only the token located exactly at the projected image coordinate can be sensitive to small calibration errors, feature-map downsampling, object boundaries, or patch discretization. To increase robustness, we also consider a local $(W \times W)$ window around each projected coordinate. A direct approach would flatten all window tokens from all cameras, leading to

$$\mathbf{I}_{s,\text{flat}}^i \in \mathbb{R}^{MW^2 \times C_s}, \quad (8)$$

and a per-point attention cost of $O(MW^2d)$.

Instead, we use a factorized formulation that separates local image refinement from multi-camera aggregation. First, for each camera m , the point query attends only to the W^2 tokens in the local image window:

$$\mathbf{I}_{s,m}^{i,\text{win}} \in \mathbb{R}^{W^2 \times C_s}. \quad (9)$$

After projecting these tokens to keys and values, the local summary for camera m is computed as

$$\mathbf{z}_{s,m}^i = \text{softmax} \left(\frac{\mathbf{Q}_s^i (\mathbf{K}_{s,m}^{i,\text{win}})^\top}{\sqrt{d}} \right) \mathbf{V}_{s,m}^{i,\text{win}} \in \mathbb{R}^{1 \times d}. \quad (10)$$

This produces one local visual summary per camera.

The camera summaries are then stacked as

$$\mathbf{Z}_s^i = [\mathbf{z}_{s,1}^i, \mathbf{z}_{s,2}^i, \dots, \mathbf{z}_{s,M}^i] \in \mathbb{R}^{M \times d}. \quad (11)$$

The point query attends over these camera-level summaries:

$$\mathbf{h}_s^i = \text{softmax} \left(\frac{\mathbf{Q}_s^i (\mathbf{Z}_s^i)^\top}{\sqrt{d}} + \mathbf{B}_s^i \right) \mathbf{Z}_s^i \in \mathbb{R}^{1 \times d}. \quad (12)$$

This factorization separates two operations: selecting relevant local evidence inside each image, and selecting the most informative camera views for the current 3D point. It therefore provides a structured way to handle small projection errors, occlusions, and viewpoint-dependent ambiguities without forcing all MW^2 image tokens to compete in a single attention operation.

Finally, we add an auxiliary geometric decoder during training. This lightweight decoder only receives point cloud features and does not use image information. It provides an additional supervision signal to the 3D encoder, encouraging the geometric branch to remain discriminative even when projected image features are missing, noisy, or uninformative. The auxiliary decoder is used as a training regularization mechanism and does not modify the main point-image fusion formulation.

IV. EXPERIMENTS

A. Datasets

We evaluate our model and its variants on several outdoor point cloud segmentation datasets: nuScenes [2], GridNet-HD [10], and two proprietary datasets provided by Helimap, corresponding to the Atocha train station and the A9CR highway. These datasets cover different acquisition settings, sensor configurations, and scene types, allowing us to evaluate the proposed fusion strategy under diverse outdoor conditions.

GridNet-HD [10] is a public multimodal dataset designed for 3D semantic segmentation of overhead electrical infrastructure. It combines high-density LiDAR point clouds with high-resolution oblique images, making it particularly relevant for LiDAR-image fusion. The dataset contains approximately 2.5 billion LiDAR points and 7,700 images acquired over power-line corridors. The scenes cover several types of environments, including mountainous, rural, field, and forest areas, and include infrastructure elements such as pylons, cables, and insulators, together with surrounding nature. The LiDAR data were acquired using a UAV creating point clouds with densities ranging from 200 to 800 points/m². The images were captured with an oblique camera configuration, which improves the visibility of lateral structures.

The original annotations are grouped into 11 semantic classes, together with an unassigned label. These classes include both infrastructure components and environmental categories. The dataset is, as many real world datasets, highly imbalanced: some classes, such as vegetation or ground-like categories, contain many more points than rare infrastructure elements. The official split separates the data into training/validation and test subsets and since the test labels are not publicly released, final test evaluation is performed through the official online benchmark.³

Helimap provided two outdoor aerial datasets with annotated LiDAR point clouds and calibrated images acquired by helicopter. The first dataset corresponds to the Atocha train station, while the

³<https://huggingface.co/datasets/heig-vg-geo/GridNet-HD>

second corresponds to the A9CR highway in Switzerland. Both datasets contain high-resolution LiDAR acquisitions and high-resolution images, including nadir and oblique views. These datasets are particularly relevant for our work because they were acquired in real-world mapping conditions and contain multiple image observations that can be associated with the same 3D points especially for the oblique view that capture diverse point of views.

Compared to GridNet-HD, the Helimap datasets contain dynamic scene elements, such as vehicles and other moving objects. This introduces additional challenges for point-image pairing and occlusion reasoning. Since the LiDAR point cloud and the images are not captured at the exact same time, a point may be projected into an image where the corresponding object is absent or where another object has moved into the line of sight. Such cases make simple projective occlusion filtering less reliable, especially when it does not rely on temporal information or high-level image understanding. These datasets therefore provide a useful test case for evaluating the robustness of image fusion under realistic acquisition artifacts.

For both Helimap datasets, we construct train, validation, and test splits following a class-distribution-aware strategy. Similar semantic labels are first merged into a common class. Then, the available spatial zones are assigned to the different splits while attempting to preserve the overall class distribution. This reduces the risk of evaluating on a split with strongly different semantic statistics from the training data. Additional details about the split construction are provided in Appendix VI.

nuScenes [2] is a large-scale multimodal autonomous driving dataset. It was collected in Boston and Singapore and contains urban driving scenes recorded under diverse conditions, including different locations, weather, lighting and traffic configurations. Each scene lasts 20 seconds, and the full dataset contains 1000 selected scenes for a total of 400k LiDAR points.

The sensor suite includes six RGB cameras, one LiDAR, five radars, GPS, and IMU measurements. The exteroceptive sensors provide full 360° coverage around the vehicle. This makes nuScenes a standard benchmark for multimodal perception, since it contains synchronized image and range-sensor data in complex urban environments. In this work, we use the LiDAR semantic segmentation benchmark associated with nuScenes to evaluate the behavior of our method on a large-scale autonomous driving dataset.

B. Metrics

We used the two metrics usually chosen to evaluate segmentation performance: *Accuracy* and *Intersection over Union* (IoU), also called the *Jaccard similarity*. All metrics are computed over the set of valid semantic classes, excluding ignored or unlabelled points.

The class-wise accuracy is defined as

$$\text{Accuracy}_c = \frac{1}{N_c} \sum_{i: y_i = c} \mathbf{1}(\hat{y}_i = c)$$

where:

- c is the class under consideration,
- y_i is the ground-truth label of point i ,
- $\hat{y}_i$ is the predicted label,
- N_c is the number of points belonging to class c ,
- $\mathbf{1}(\cdot)$ is the indicator function.

As we can observe, this metric does not directly penalize points that are predicted as class c while actually belonging to another class.

However, IoU is generally preferred in segmentation tasks, especially under highly imbalanced class distributions, because it jointly accounts for both false positives and false negatives, preventing dominant classes from artificially inflating performance metrics. In contrast, accuracy can be biased toward frequent classes, whereas IoU provides a more balanced evaluation across classes by measuring the overlap between predicted and ground-truth regions.

The IoU is therefore more conservative and more penalizing. As its name indicates, it is computed as the intersection divided by the union of two sets. In semantic segmentation, the intersection corresponds to the number of correctly predicted elements for class c , while the union contains:

- the correctly predicted elements,
- the false positives,
- and the false negatives.

Mathematically, the IoU for class c is defined as

$$\text{IoU}_c = \frac{TP_c}{TP_c + FP_c + FN_c}$$

where:

- TP_c denotes the number of true positives,
- FP_c the number of false positives,
- FN_c the number of false negatives.

Equivalently, using set notation:

$$\text{IoU}_c = \frac{|P_c \cap G_c|}{|P_c \cup G_c|}$$

where P_c is the set of points predicted as class c and G_c is the set of ground-truth points belonging to class c .

For completeness, we also reported the overall accuracy, which is computed by summing all correct classified points divided by the number of valid points. This metric is also biased by unbalanced dataset but is sometimes given.

C. Implementation Details

Data Preprocessing We follow the preprocessing and training pipeline of Pointcept [45] and DINO in the Room [31] for nuScenes. For GridNet-HD and the Helimap datasets, the original scenes are divided into spatial chunks before training and inference. For GridNet-HD and Atocha, we use chunks of size $18 \times 18 \text{ m}^2$ in the x-y plane with a stride of 9 m. This overlapping strategy allows the model to process local regions while preserving sufficient spatial context. During inference, overlapping predictions are merged by summing the logits of all chunks covering a point, followed by a final arg max operation. For A9CR, the point density and scene size lead to a substantially larger number of points per chunk. We therefore reduce the chunk size to $14 \times 14 \text{ m}^2$, which makes training and inference tractable while keeping enough local context for segmentation.

Image Preprocessing For GridNet-HD, we downsample the original images from 3956×5280 pixels to 768×1008 pixels while approximately preserving the aspect ratio. This resolution provides a practical compromise between image detail and GPU memory usage. The Helimap images are substantially larger, with resolutions of 11664×8750 pixels for the front cameras and 9564×12768 pixels for the nadir cameras. Processing these images at full resolution is not tractable, while directly downsampling them to a small fixed resolution would remove many fine visual details and degrade the overall quality of the image.

To address this, we use a chunk-based image cropping strategy for the Helimap datasets. For each point cloud chunk, we project the corresponding 3D points into the image and compute the image-space bounding box covering the projected coordinates. Because a small number of noisy or outlier projections can artificially enlarge the bounding box, we remove extreme projected coordinates using percentile-based filtering. The resulting bounding box is then expanded by a small margin to include additional context and to avoid placing projected points exactly on crop boundaries. The crop is finally resized to a resolution compatible with the image encoder. To avoid unnecessary upsampling of very small crops, we enforce a minimal crop size before interpolation. The final image size is chosen to be 1040×1040 .

For the image branch, images are first resized or cropped to the desired resolution while ensuring that their spatial dimensions are divisible by the patch size of the selected image encoder, which is 14 or 16 depending on the DINO model. During training, we apply image augmentations such as color jittering and random horizontal flipping. The images are then normalized using the mean and standard deviation associated with the pretrained image backbone.

This preprocessing makes it possible to preserve local image details while keeping the computational cost manageable. It also allows the visual information to remain spatially aligned with the point cloud

chunk used by the 3D backbone. Since DINOv3 models are trained with stronger robustness to varying image resolutions, they are well suited to this cropping strategy. For DINOv2, whose public checkpoints are provided from models partially trained on 518×518 input resolution [20], very large input resolutions may lead to less reliable feature behavior. For this reason, the final experiments on our modules mainly rely on DINOv3.

Training Configuration We use a voxel resolution of 0.05 m for the outdoor datasets. This resolution provides a compromise between geometric detail and memory consumption. It is further justified and consistent with the observations reported for GridNet-HD, where reducing the resolution to 0.1 m decreases segmentation performance [10]. During training, we apply standard point cloud augmentations, including random rotations around the vertical axis, random scaling, jittering, and flipping. The main backbone is trained with a learning rate of 2×10^{-3} , while self-attention and cross-attention blocks use a lower learning rate of 2×10^{-4} .

We chose to train/test at resolution of 0.05m. This was further justified in [10] where they showed that reduce the resolution to 0.1m worsened the predictions. We applied random rotation around the z axis, scaling, jitter and flip. We used a learning rate of 2×10^{-3} and 2×10^{-4} for the self and cross attention blocks.

We also use Mix3D [46] as a data augmentation strategy. Mix3D constructs a new training sample by concatenating two point clouds and their corresponding labels. Although the resulting scene may not correspond to a physically realistic environment, this augmentation intentionally disrupts global scene context. As a consequence, the model is discouraged from relying too strongly on dataset-specific global layouts and is encouraged to learn more local geometric. This is particularly relevant for large-scale scenes, where models may otherwise exploit contextual shortcuts instead of learning robust point-level representations.

Loss Function The model is trained using a combination of cross-entropy loss and Lovász-Softmax loss [47]. Cross-entropy provides stable point-wise supervision, while the Lovász-Softmax loss is a differentiable surrogate designed to optimize the mean IoU more directly. For our multimodal model, we additionally use an auxiliary geometric decoder during training. This decoder only receives point cloud features and is supervised with the same combination of cross-entropy and Lovász-Softmax losses. The auxiliary loss is weighted by a coefficient λ_{aux} . In our experiments, setting $\lambda_{\text{aux}} = 0.5$ provides a small improvement, while increasing it to 1.0) leads to less stable training. Therefore, we use $\lambda_{\text{aux}} = 0.5$ in the final configuration.

Inference At inference time, we apply test-time augmentation (TTA) [48]. We use geometric transformations such as scaling and flipping, run the model on each transformed version of the input, and aggregate the resulting logits before computing the final prediction. This provides more stable predictions on unseen scenes and reduces sensitivity to a particular input orientation or scale.

D. Architecture Details

Image Encoder For the DITR baseline used in our comparisons, we use a DINOv2 image encoder. Following the original DITR configuration, we use the giant Vision Transformer variant (ViT-g), as this model was reported to perform better on outdoor datasets [31]. The remaining architectural hyperparameters, including the 3D decoder dimensions, are kept consistent with the original DITR setup.

For our proposed model, we use the large DINOv3 image encoder (ViT-L). This variant was selected because DINOv3 is designed to produce stable visual embeddings across a wide range of image resolutions, including very high-resolution inputs [21]. This property is particularly useful for our setting, where aerial images may be substantially large.

Multi-Layer Feature Extraction For DINOv3 ViT-L, we select four intermediate layers uniformly across the network depth. In practice, we use layers 5, 11, 17, and 23, ordered from shallow to deep. Each selected feature map is projected to a lower-dimensional representation with $C_{\text{proj}} = 128$. After concatenating the four projected feature maps, the resulting feature dimension is $4 \cdot 128 = 512$, which remains manageable for both training and inference. We also considered $C_{\text{proj}} = 256$, but this setting often led to out-of-memory errors on GridNet-HD and Helimap datasets. Therefore, we use $C_{\text{proj}} = 128$ in the final configuration.

3D Backbone The remaining architectural hyperparameters of the DITR baseline, including the 3D decoder dimensions, are kept consistent with the original DITR setup. For our proposed model, the 3D branch retains the encoder-decoder dimensions from the original PTv3 and LitePT-S configurations.

Multi-Camera Attention Module We apply the factorized multi-camera attention module at the two coarsest decoder stages. Although cross-attention over camera observations is relatively lightweight for a small number of points, its memory and latency increase substantially at finer decoder stages, where the number of points is much larger. For the remaining decoder stages, we therefore use the additive fusion formulation of Eq. 4, following [31]. At these stages, when a point is associated with multiple valid image observations, we randomly sample one image feature during training and use the first valid image feature during evaluation.

For the factorized attention module, we use a local window size of 3×3 , corresponding to 9 image tokens around the projected point location at the coarsest stage and a single token at the other stage. The coarsest decoder stage uses 6 attention heads, while the second coarsest stage uses 4 attention heads. A feed-forward network is applied after the attention operation to further refine the fused representation.

We observed that using the cross-attention mechanism didn’t work well with the pyramid modules and thus we kept the original formulation from [31] for this module.

Residual Scaling To improve training stability, the output of the image-attention module is injected through a learnable residual scaling parameter initialized to zero. This prevents strong image features from being injected too early in training and allows the network to progressively learn how much visual information should be used. The update is written as

$$\mathbf{o}_i = \mathbf{b}_i + \lambda \odot \mathbf{y}_i,$$

where $\mathbf{b}_i$ denotes the input point feature, $\mathbf{y}_i$ is the output of either the flat attention module $\mathbf{y}_i^{\text{flat}}$ or the factorized attention module $\mathbf{y}_i^{\text{fact}}$, and λ is a learnable channel-wise scale.

Auxiliary Decoder and Segmentation Head The auxiliary geometric decoder uses the same channel dimensions as the main decoder. It follows a lightweight unpooling design in which the skip features and the unpooled coarse features are first projected with linear layers and then summed. Layer normalization is applied after the summation. This auxiliary head is used only during training and provides an additional geometry-only supervision signal.

Finally, a linear segmentation head is applied at the end of the decoder to produce the final predictions

E. Results

We compare the proposed architectures against recent point-cloud and multimodal segmentation methods. In the following, we refer to the lightweight multi-layer concatenation architecture as Concat-LitePT and to the spatial pyramid architecture as Pyramid-LitePT.

1) *nuScenes*: Table I reports the mIoU on the nuScenes validation set. The methods are separated into three groups: point-cloud-only methods, existing multimodal methods, and our proposed architectures. Both proposed models improve over the previous multimodal baseline DITR. Concat-LitePT reaches 84.4% mIoU, corresponding to a gain of +0.2% over DITR, while Pyramid-LitePT reaches 84.6%, improving over DITR by +0.4%. Although the improvement is moderate, it indicates that replacing the original 3D backbone with LitePT and incorporating multi-layer image features can slightly improve the performance on a large-scale autonomous driving benchmark.

Method	nuScenes
PTv3 [15]	80.4
Sonata [25]	81.7
LitePT	82.2
Utonia [27]	82.2
MSeg3D [49]	80.0
2DPASS [50]	79.4
LCPS [18]	80.5
DITR [31]	84.2
Concat-LitePT	84.4
Pyramid-LitePT	84.6

TABLE I: mIoU comparison on the nuScenes validation set.

2) *GridNet-HD*: Table II reports the results on the GridNet-HD test set. In addition to comparing against the published baselines, we reproduce LitePT-S on GridNet-HD in order to evaluate the strength of the 3D backbone alone. LitePT-S improves over PTv3 by +3.56% mIoU, showing that the lightweight backbone is already a strong baseline on this dataset. DITR further improves over LitePT-S, confirming the benefit of adding image information on GridNet-HD.

Our proposed architectures improve over both the 3D-only baselines and DITR. Concat-LitePT obtains the best result, with 78.25% mIoU and 87.4% mAcc. This corresponds to an improvement of +3.38% mIoU and +0.37% mAcc over DITR. Pyramid-LitePT also improves over DITR in terms of mIoU, reaching 77.68%, but remains slightly below Concat-LitePT. These results suggest that multi-layer image features are beneficial on GridNet-HD, and that the simpler concatenation strategy is more effective than the spatial pyramid variant in this setting.

Method	mIoU (%)	mAcc (%)
SPT [51]	66.90	86.82
PTv3	69.32	86.05
LitePT [†]	72.88	86.32
DITR	74.87	87.03
Concat-LitePT	78.25	87.4
Pyramid-LitePT	77.68	87

TABLE II: Performance on GridNet. Metrics are reported in %. [†] Result reproduced in this work using the official LitePT implementation and training protocol.

3) *Helimap Datasets*: Table III reports the results on the two Helimap datasets, Atocha and A9CR. The results show that the effect of image fusion depends strongly on the dataset.

Atocha On Atocha, Concat-LitePT achieves the best performance on both validation and test sets, reaching 82.0% and 84.6% mIoU, respectively. On the test set, this represents an improvement of +2.0% over LitePT-S and +6.1% over DITR. The improvement is also visible in mAcc and OA, where Concat-LitePT obtains the best or near-best values. This indicates that the visual features are useful on Atocha and that the proposed fusion strategy can improve over both 3D-only and existing multimodal baselines.

Method	Atocha		A9CR	
	Val	Test	Val	Test
PTv3	76.8	82.2	75.4	72.9
LitePT-S	78.2	82.6	78.1	78.9
DITR	73.1	78.5	74.8	71.3
LitePT-concat	82.0	84.6	79.4	70.5
LitePT-pyramid	75.3	81.1	75.7	71.6

(a) mIoU

Method	Atocha				A9CR			
	mAcc		OA		mAcc		OA	
	Val	Test	Val	Test	Val	Test	Val	Test
PTv3	85.9	90.7	94.3	96.6	84.3	83.7	95.3	95.4
LitePT-S	87.8	92.1	93.5	94.9	85.0	85.5	96.0	96.3
DITR	82.0	86.9	92.5	95.6	82.2	84.1	94.5	94.4
LitePT-concat	89.0	93.6	94.4	96.8	89.0	84.0	95.8	93.3
LitePT-pyramid	85.4	89.8	93.8	96.1	83.0	81.5	93.5	93.9

(b) mAcc and OA

TABLE III: Performance comparison on the Helimap datasets. Metrics are reported in %.

Pyramid-LitePT behaves differently. On Atocha, it improves over DITR but remains below the 3D-only baselines and below Concat-LitePT. This suggests that the spatial pyramid module does not consistently exploit the visual features better than the simpler concatenation module. One possible explanation is that the additional pyramid structure introduces more parameters and more transformations of the image features, which may make optimization harder or increase sensitivity to imperfect point-image correspondences.

A9CR The behavior on A9CR is more challenging. On the validation set, Concat-LitePT obtains the best mIoU, reaching 79.4%, slightly above LitePT-S. However, on the test set, both image-assisted variants underperform the LitePT-S baseline. LitePT-S reaches 78.9% mIoU, while Concat-LitePT and Pyramid-LitePT obtain 70.5% and 71.6%, respectively. This discrepancy between validation and test suggests that the image branch may be less robust on A9CR, possibly due to stronger domain shift, class imbalance, dynamic objects, projection noise, or less reliable image-point correspondences.

a) Per-class analysis.: The per-class IoU tables provide more insight into these differences. On Atocha test, Concat-LitePT improves several important classes compared to LitePT-S, including ground, building, wall, rail, and mobile objects. The gain on wall is particularly large, suggesting that image appearance can help distinguish vertical structures that may be geometrically ambiguous. However, some classes such as fence and lightpole do not improve, indicating that image fusion is not uniformly beneficial for all categories.

Model	Ground	Vegetation	Building	Wall	Fence	Rail	Rail catenary	Pylon rail	Lightpole	Mobile object	mIoU
PTv3	86.78	98.91	96.68	52.92	66.35	80.51	97.71	96.98	62.86	82.39	82.21
LitePT-S	77.16	99.01	93.94	54.92	66.57	80.79	97.45	97.06	73.54	85.83	82.63
DITR	79.8	98.2	96.0	61.9	51.8	76.3	95.9	93.8	47.8	83.8	82.6
Concat-LitePT	85.8	98.5	96.8	65.7	61.1	84.0	97.9	96.5	72.0	87.2	84.6
Pyramid-LitePT	83.2	96.0	96.4	68.7	43.5	80.6	96.7	95.8	64.7	85.3	81.1

TABLE IV: IoU per class for each model on Atocha test set

On A9CR test, the drop in mIoU is mainly caused by a few classes. Compared to LitePT-S, Concat-LitePT strongly decreases on vegetation, net fence animal, low point, and especially airpoint. The airpoint class is the most severe failure case, dropping from 82.6% with LitePT-S to 10.8% with Concat-LitePT. Since mIoU averages all classes equally, this single class contributes substantially to the overall decrease. At the same time, Concat-LitePT improves some classes, such as noise fence and mobile objects, showing that image information is not globally harmful but may be unreliable for specific categories.

Model	Ground	Vege.	Build.	Safety fence	Net fence animal	Noise fence	Road furniture	Power cable	Power supp.	Mob. obj.	Low point	Airpoint	mIoU
PTv3	97.3	86.7	25.9	93.3	57.0	51.1	87.4	94.1	81.9	82.9	71.1	45.8	72.9
LitePT-S	98.0	88.8	29.2	92.7	58.7	64.3	91.1	92.6	86.8	84.1	77.3	82.6	78.9
DITR	97.3	83.3	22.3	92.6	50.6	71.7	86.4	93.5	86.3	83.8	65.9	21.3	71.3
Concat-LitePT	97.1	77.5	25.3	93.1	46.6	74.8	90.6	93.4	84.4	87.0	65.9	10.8	70.5
Pyramid-LitePT	95.6	84.8	26.5	93.6	50.2	73.0	88.8	90.1	84.3	89.4	50.7	32.6	71.6

TABLE V: IoU per class for each model on A9CR test set

4) Summary: Overall, the results show that Concat-LitePT is the most robust proposed variant across the evaluated datasets. It improves over DITR on nuScenes and GridNet-HD, and gives the best results on Atocha. However, the failure on A9CR test highlights that point-image fusion remains sensitive to

dataset-specific conditions and to the quality of the projected image features. These results suggest that future work should further investigate visibility filtering, dynamic-object handling, and class-dependent effects of image fusion.

V. LIMITATIONS

Although the proposed models show promising results, several limitations remain. First, integrating intermediate image features increases the memory cost of the architecture, especially for the pyramid variant, which processes several feature maps at different resolutions. This is particularly limiting for high-density datasets such as GridNet-HD and the Helimap datasets, where each chunk can contain a large number of points. In practice, the use of a lightweight 3D backbone such as LitePT is important to keep the full multimodal model tractable.

Second, the spatial pyramid module does not consistently improve over the simpler concatenation module. While the pyramid design is motivated by the multi-scale structure of the 3D decoder, it introduces additional transformations and stage-specific feature maps, which may make optimization harder and increase sensitivity to imperfect point-image correspondences.

Finally, the method remains dependent on the quality of point-image pairing. Projection errors, occlusions, moving objects, and temporal inconsistencies between LiDAR and image acquisition can introduce noisy visual features. This is especially visible on A9CR, where image fusion improves some classes but degrades others. The multi-camera attention module provides a more structured aggregation of image observations, but it remains too expensive to apply at all decoder stages on very dense point clouds.

VI. CONCLUSION

In this work, we studied image-assisted point cloud semantic segmentation using frozen vision foundation model features and lightweight 3D backbones. We proposed two image feature modules that exploit intermediate layers of a DINO image encoder: a lightweight concatenation module and a spatial pyramid module. We also investigated a factorized multi-camera attention mechanism designed to aggregate several image observations associated with the same 3D point.

The experiments show that intermediate image features can provide useful complementary information to the 3D backbone. On nuScenes and GridNet-HD, the proposed architectures improve over previous multimodal baselines, with the strongest gains observed on GridNet-HD. On the Helimap datasets, the results are more contrasted. Concat-LitePT achieves strong improvements on Atocha and obtains the best performance on several metrics, but both image-assisted variants underperform the 3D-only LitePT-S baseline on the A9CR test set. This highlights that image fusion is not always beneficial and strongly depends on the quality of the point-image correspondences, the dataset characteristics, and the reliability of the visual observations.

Among the proposed variants, the concatenation module appears to be the most robust. Despite its simplicity, it consistently provides competitive or improved performance while requiring less memory than the pyramid variant. The spatial pyramid module is theoretically appealing because it aligns image features with the multi-scale hierarchy of the 3D decoder, but in practice it introduces additional complexity and does not consistently improve the results. Similarly, the multi-camera attention mechanism provides a structured way to aggregate several views, but remains costly on very dense point clouds and is therefore limited to coarse decoder stages.

Overall, this work suggests that frozen image foundation models can improve point cloud segmentation when their features are carefully integrated into the 3D architecture. However, the results also show that multimodal fusion remains sensitive to dataset-specific conditions, especially in large-scale real-world acquisitions with occlusions, dynamic objects, and imperfect alignment. Future work should therefore focus on more robust visibility filtering, reliability-aware image feature selection, and efficient multi-view aggregation mechanisms that can scale to high-resolution point clouds.

REFERENCES

- [1] Y. He, H. Yu, X. Liu, Z. Yang, W. Sun, S. Anwar, and A. Mian, “Deep learning based 3d segmentation: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2103.05423>
- [2] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, “nusenes: A multimodal dataset for autonomous driving,” 2020. [Online]. Available: <https://arxiv.org/abs/1903.11027>
- [3] P. Sun, H. Kretschmar, X. Dotiwala, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, V. Vasudevan, W. Han, J. Ngiam, H. Zhao, A. Timofeev, S. Ettinger, M. Krivokon, A. Gao, A. Joshi, S. Zhao, S. Cheng, Y. Zhang, J. Shlens, Z. Chen, and D. Anguelov, “Scalability in perception for autonomous driving: Waymo open dataset,” 2020. [Online]. Available: <https://arxiv.org/abs/1912.04838>
- [4] T. Zhang, Z. Liang, and B. Wang, “A survey of medical point cloud shape learning: Registration, reconstruction and variation,” in *2025 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, Dec. 2025, p. 5777–5782. [Online]. Available: <http://dx.doi.org/10.1109/BIBM66473.2025.11357130>
- [5] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “Pointnet: Deep learning on point sets for 3d classification and segmentation,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 652–660.
- [6] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “Pointnet++: Deep hierarchical feature learning on point sets in a metric space,” *Advances in neural information processing systems*, vol. 30, 2017.
- [7] D. Halperin and N. Eisl, “Point cloud based scene segmentation: a survey,” *arXiv preprint arXiv:2503.12595*, 2025.
- [8] A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner, “ScanNet: Richly-annotated 3d reconstructions of indoor scenes,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 5828–5839.
- [9] I. Armeni, O. Sener, A. R. Zamir, H. Jiang, I. Brilakis, M. Fischer, and S. Savarese, “3d semantic parsing of large-scale indoor spaces,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 1534–1543.
- [10] A. Carreaud, S. Li, M. De Lacour, D. Frinde, J. Skalous, and A. Gressin, “Gridnet-hd: A high-resolution multi-modal dataset for lidar-image fusion on power line infrastructure,” *arXiv preprint arXiv:2601.13052*, 2026.
- [11] C. Choy, J. Gwak, and S. Savarese, “4d spatio-temporal convnets: Minkowski convolutional neural networks,” 2019. [Online]. Available: <https://arxiv.org/abs/1904.08755>
- [12] B. Graham, M. Engelcke, and L. van der Maaten, “3d semantic segmentation with submanifold sparse convolutional networks,” 2017. [Online]. Available: <https://arxiv.org/abs/1711.10275>
- [13] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, “Dynamic graph cnn for learning on point clouds,” *ACM Transactions on Graphics (tog)*, vol. 38, no. 5, pp. 1–12, 2019.
- [14] H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas, “Kpconv: Flexible and deformable convolution for point clouds,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 6411–6420.
- [15] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, and H. Zhao, “Point transformer v3: Simpler faster stronger,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 4840–4851.
- [16] Y. Yue, D. Robert, J. Wang, S. Hong, J. D. Wegner, C. Rupprecht, and K. Schindler, “Litept: Lighter yet stronger point transformer,” *arXiv preprint arXiv:2512.13689*, 2025.
- [17] Z. Duan, X. Hu, P. An, and J. Ma, “Multimodal point cloud semantic segmentation with virtual point enhancement,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.01449>
- [18] Z. Zhang, Z. Zhang, Q. Yu, R. Yi, Y. Xie, and L. Ma, “Lidar-camera panoptic segmentation via geometry-consistent and semantic-aware alignment,” 2023. [Online]. Available: <https://arxiv.org/abs/2308.01686>
- [19] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging properties in self-supervised vision transformers,” 2021. [Online]. Available: <https://arxiv.org/abs/2104.14294>
- [20] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby *et al.*, “Dinov2: Learning robust visual features without supervision,” *arXiv preprint arXiv:2304.07193*, 2023.
- [21] O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, C. Jose, V. Khalidov, M. Szafraniec, S. Yi, M. Ramamonjisoa *et al.*, “Dinov3,” *arXiv preprint arXiv:2508.10104*, 2025.
- [22] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, “Segment anything,” 2023. [Online]. Available: <https://arxiv.org/abs/2304.02643>
- [23] D. Robert, B. Vallet, and L. Landrieu, “Learning multi-view aggregation in the wild for large-scale 3d semantic segmentation,” 2022. [Online]. Available: <https://arxiv.org/abs/2204.07548>
- [24] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” 2015. [Online]. Available: <https://arxiv.org/abs/1505.04597>
- [25] X. Wu, D. DeTone, D. Frost, T. Shen, C. Xie, N. Yang, J. Engel, R. Newcombe, H. Zhao, and J. Straub, “Sonata: Self-supervised learning of reliable point representations,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 22 193–22 204.
- [26] Y. Zhang, X. Wu, Y. Lao, C. Wang, Z. Tian, N. Wang, and H. Zhao, “Concerto: Joint 2d-3d self-supervised learning emerges spatial representations,” *arXiv preprint arXiv:2510.23607*, 2025.
- [27] Y. Zhang, X. Wu, Y. Yang, X. Fan, H. Li, Y. Zhang, Z. Huang, N. Wang, and H. Zhao, “Utonia: Toward one encoder for all point clouds,” *arXiv preprint arXiv:2603.03283*, 2026.
- [28] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “Roformer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024.
- [29] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” 2015. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [30] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” 2021. [Online]. Available: <https://arxiv.org/abs/2010.11929>
- [31] K. Knaebel, K. Yilmaz, D. de Geus, A. Hermans, D. Adrian, T. Linder, and B. Leibe, “Dino in the room: Leveraging 2d foundation models for 3d segmentation,” *arXiv preprint arXiv:2503.18944*, 2025.

- [32] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, “Dust3r: Geometric 3d vision made easy,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.14132>
- [33] J. Wang, M. Chen, S. Zhang, N. Karaev, J. Schönberger, P. Labatut, P. Bojanowski, D. Novotny, A. Vedaldi, and C. Rupprecht, “Vgggt- ω ,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.15195>
- [34] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, “Depth anything v2,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.09414>
- [35] S. Amir, Y. Gandelsman, S. Bagon, and T. Dekel, “Deep vit features as dense visual descriptors,” 2022. [Online]. Available: <https://arxiv.org/abs/2112.05814>
- [36] R. Ranftl, A. Bochkovskiy, and V. Koltun, “Vision transformers for dense prediction,” 2021. [Online]. Available: <https://arxiv.org/abs/2103.13413>
- [37] S. Yang, H. Wang, Z. Xing, S. Chen, and L. Zhu, “Segdino: An efficient design for medical and natural image segmentation with dino-v3,” *arXiv preprint arXiv:2509.00833*, 2025.
- [38] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” 2017. [Online]. Available: <https://arxiv.org/abs/1612.03144>
- [39] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, “Multimodal machine learning: A survey and taxonomy,” 2017. [Online]. Available: <https://arxiv.org/abs/1705.09406>
- [40] D. Hendrycks and K. Gimpel, “Gaussian error linear units (gelus),” 2023. [Online]. Available: <https://arxiv.org/abs/1606.08415>
- [41] J. P. Schwarz Schuler, S. R. Also, D. Puig, H. Rashwan, and M. Abdel-Nasser, “An enhanced scheme for reducing the complexity of pointwise convolutions in cnns for image classification based on interleaved grouped filters without divisibility constraints,” *Entropy*, vol. 24, no. 9, p. 1264, 2022.
- [42] L. Kaiser, A. N. Gomez, and F. Chollet, “Depthwise separable convolutions for neural machine translation,” *arXiv preprint arXiv:1706.03059*, 2017.
- [43] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A convnet for the 2020s,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 11 976–11 986.
- [44] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [45] P. Contributors, “Pointcept: A codebase for point cloud perception research,” <https://github.com/Pointcept/Pointcept>, 2023.
- [46] A. Nekrasov, J. Schult, O. Litany, B. Leibe, and F. Engelmann, “Mix3d: Out-of-context data augmentation for 3d scenes,” 2021. [Online]. Available: <https://arxiv.org/abs/2110.02210>
- [47] M. Berman, A. R. Triki, and M. B. Blaschko, “The lovasz-softmax loss: A tractable surrogate for the optimization of the intersection-over-union measure in neural networks,” 2018. [Online]. Available: <https://arxiv.org/abs/1705.08790>
- [48] M. Kimura, “Understanding test-time augmentation,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.06892>
- [49] J. Li, H. Dai, H. Han, and Y. Ding, “Mseg3d: Multi-modal 3d semantic segmentation for autonomous driving,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.08600>
- [50] X. Yan, J. Gao, C. Zheng, C. Zheng, R. Zhang, S. Cui, and Z. Li, “2dpas: 2d priors assisted semantic segmentation on lidar point clouds,” 2022. [Online]. Available: <https://arxiv.org/abs/2207.04397>
- [51] D. Robert, H. Raguét, and L. Landrieu, “Efficient 3d semantic segmentation with superpoint transformer,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.08045>

APPENDIX

Detailed Reprojection Formulation

This appendix reports the full camera projection formulation used in [10]. While the main text presents the standard pinhole camera model, the formulation below details the Agisoft Metashape camera convention, including Euler-angle rotations, lens distortion correction, and final conversion to pixel coordinates. We keep the notation consistent with the main text where possible.

Let a 3D point in world coordinates be denoted by

$$P_w = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}.$$

The camera orientation is parameterized by the Euler angles (ω, ϕ, κ) . Following the Agisoft Metashape convention, the elementary rotation matrices are

$$R_x(\omega) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \omega & \sin \omega \\ 0 & -\sin \omega & \cos \omega \end{bmatrix},$$

$$R_y(\phi) = \begin{bmatrix} \cos \phi & 0 & -\sin \phi \\ 0 & 1 & 0 \\ \sin \phi & 0 & \cos \phi \end{bmatrix},$$

and

$$R_z(\kappa) = \begin{bmatrix} \cos \kappa & \sin \kappa & 0 \\ -\sin \kappa & \cos \kappa & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The full rotation matrix is then defined as

$$R = R_z(\kappa)R_y(\phi)R_x(\omega).$$

Let the camera center in world coordinates be

$$S = \begin{bmatrix} X_s \\ Y_s \\ Z_s \end{bmatrix}.$$

The world point P_w is transformed into camera coordinates as

$$P_c = \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = R(P_w - S).$$

Equivalently, in homogeneous coordinates, this can be written as

$$P_c = [R \mid t] \hat{P}_w,$$

where

$$\hat{P}_w = \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}, \quad t = -RS.$$

Following the Agisoft convention, normalized image coordinates are computed as

$$x = -\frac{X_c}{Z_c}, \quad y = -\frac{Y_c}{Z_c}, \quad z = -Z_c.$$

Here, x and y denote normalized image-plane coordinates, while z is the depth value used for visibility reasoning.

Lens distortion is modeled using radial and tangential distortion coefficients

$$(k_1, k_2, k_3, k_4, k_5, p_1, p_2, p_3, p_4).$$

The squared radial distance is

$$r_c = x^2 + y^2.$$

The radial distortion term is given by

$$d_r = 1 + k_1 r_c + k_2 r_c^2 + k_3 r_c^3 + k_4 r_c^4 + k_5 r_c^5.$$

The tangential distortion terms are

$$d_{t,x} = p_1(r_c + 2x^2) + 2p_2xy(1 + p_3r_c + p_4r_c^2),$$

$$d_{t,y} = p_2(r_c + 2y^2) + 2p_1xy(1 + p_3r_c + p_4r_c^2).$$

The distorted normalized coordinates are then

$$x' = xd_r + d_{t,x}, \quad y' = yd_r + d_{t,y}.$$

Let W and H denote the image width and height, respectively. Let f be the focal length, (c_x, c_y) the principal point offsets, and (b_1, b_2) the sensor skew coefficients. The final pixel coordinates are obtained as

$$u = \frac{W}{2} + c_x + x'f + x'b_1 + y'b_2,$$

$$v = \frac{H}{2} + c_y + y'f.$$

Thus, the projection function can be written as

$$\pi(P_w) = (u, v).$$

Helimap dataset split

Helimap provided 2 different scenes, the train station of Atocha and the A9 highway. The data comprises several LAZ files containing the coordinates of the 3D points, their colours and their labels. The images are supplied with their camera calibrations (pose and optical correction).

We had first to split the 2 datasets in 3 splits, the train, evaluation and test set. Like the majority of real world datasets, the classes are unbalanced and some classes are rare and this can make the separation difficult. For efficiency and for simplicity, we only splitted the laz files and removed the classes that appear in less than 3 files because they had to appear in the 3 splits set for training and testing consistency.

The train/validation/test split is built from the class distribution of each .laz or .las file. Each file is loaded, its labels are remapped to the dataset semantic classes, and a histogram is computed to count how many points of each class it contains. Ignored or unlabeled classes are excluded from the scoring, so they do not influence the split quality.

We can summarize the algorithm of the 3 split sets like this:

The goal is then to assign each file to train, validation, or test while keeping the class distributions close to the desired ratios, for example 60/20/20. The score measures, for each class, how far the fraction of points assigned to each split is from the target ratio. Rare classes are given more weight, so that the split does not only optimize the frequent classes. An additional coverage constraint ensures that each split receives at least a minimum fraction of every class, preventing rare classes from disappearing from validation or test.

For a small number of files, the algorithm tests all possible assignments and keeps the one with the best score. For larger datasets, where testing all (3^F with F the number of files) possibilities would be too expensive, it uses a heuristic search. It first builds a deterministic greedy split by placing the largest files and then assigning the remaining ones where they improve the score the most. This split is refined by moving files between sets or swapping files across sets whenever this improves the objective. To avoid being stuck in a poor local solution, the same refinement is also applied several times from random initial splits, and the best solution found is kept.

TABLE VI: Global statistics of the train/validation/test splits. Ignored points correspond to labels excluded from the split scoring because they are considered unlabeled.

Dataset	Files	Total points	Ignored points	Ignored
Atocha	17	614 069 295	314 302 315	51.2%
A9 highway	73	6 569 830 747	5 663 735 004	86.2%

TABLE VII: Statistics per split. The target ratio is 60/20/20 for train/validation/test. “Scored ratio” is computed only over points used for scoring, excluding ignored labels.

Dataset	Split	Files	Total points	Scored points	Scored ratio	Ignored
Atocha	Train	11	377 387 117	176 048 480	58.7%	53.4%
	Validation	2	136 880 275	80 095 694	26.7%	41.5%
	Test	4	99 801 903	43 622 806	14.6%	56.3%
A9CR	Train	43	3 928 267 854	534 686 136	59.0%	86.4%
	Validation	14	1 258 137 403	185 664 731	20.5%	85.2%
	Test	16	1 383 425 490	185 744 876	20.5%	86.6%

TABLE VIII: Class-wise coverage for the Atocha split. Values indicate the percentage of each class assigned to train, validation, and test.

Class	Total points	Train	Validation	Test
ground	24 944 649	48.2%	27.7%	24.1%
vegetation	33 328 308	77.9%	11.0%	11.2%
building	203 744 498	56.4%	30.4%	13.2%
wall	3 264 646	59.1%	27.7%	13.2%
fence	1 924 404	51.2%	38.4%	10.4%
rail	7 558 827	66.7%	10.6%	22.7%
rail catenary	6 119 973	64.0%	14.9%	21.1%
pylon rail	7 172 759	66.0%	12.7%	21.3%
lightpole	197 458	65.7%	16.2%	18.1%
mobile object	11 511 458	56.5%	28.6%	15.0%

TABLE IX: Class-wise coverage for the A9CR split. Values indicate the percentage of each class assigned to train, validation, and test.

Class	Total points	Train	Validation	Test
ground	601 076 778	58.7%	20.1%	21.2%
vegetation	165 661 213	56.9%	23.5%	19.6%
building	23 209 833	51.6%	30.8%	17.6%
safety fence	36 776 201	58.5%	20.8%	20.7%
net fence animal	4 659 190	60.3%	19.9%	19.9%
noise fence	3 465 161	60.3%	20.0%	19.7%
road furniture	2 748 745	59.9%	20.1%	20.0%
power cable	907 353	58.6%	19.2%	22.2%
power support	573 988	60.9%	12.0%	27.2%
mobile object	14 655 450	65.6%	20.5%	13.9%
low point	50 099 256	71.2%	10.1%	18.7%
airpoint	2 262 575	60.0%	20.0%	20.0%

Extra figure

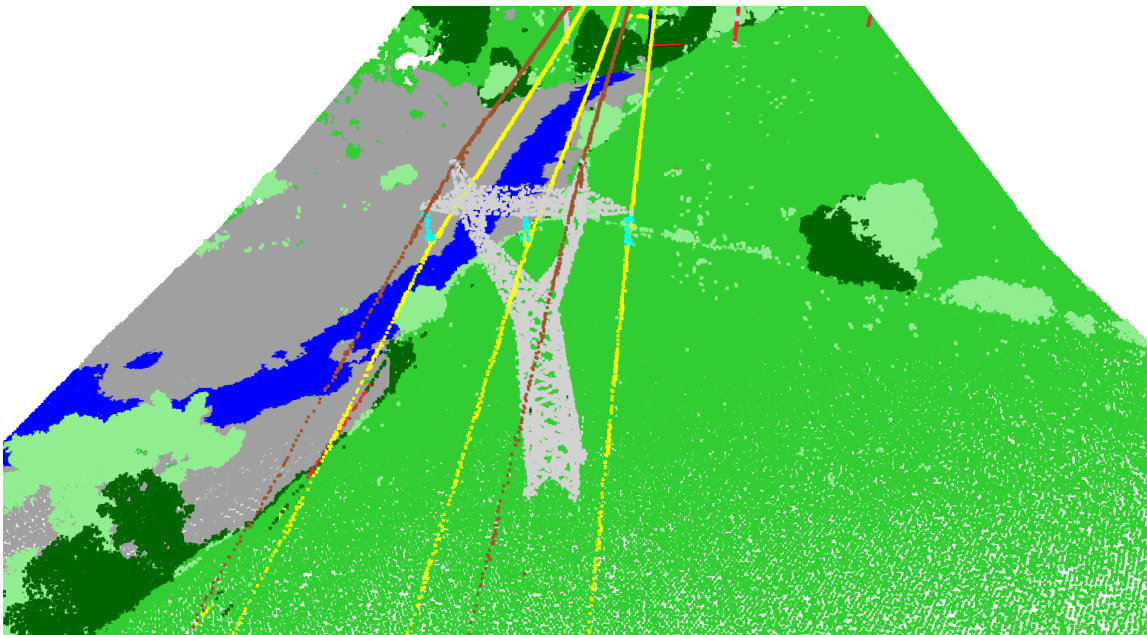


Fig. 4: Example of results from GridNet test set with Concat-LitePT